

Deep Reinforcement Learning (DQN)

Fundamentals · Hands-on examples · Python Implementation

CSE4020 — Capstone Design | Dr. Oybek Eraliev

Table of Contents

What we will cover in this lecture

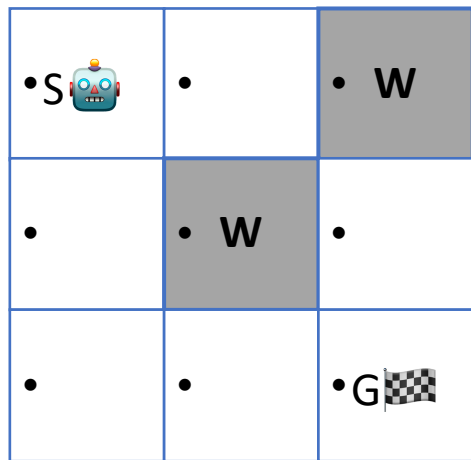
- **01** Recap: Q-Learning
- **02** DQN Architecture: Neural Network as Function Approximator
- **03** Full DQN Algorithm
- **04** DQN Implementation using Python

Recap: Q-Learning

Hand Calculation — Step by Step

Initial Q-Table (all zeros) Parameters: $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.3$

State	↑ Up	↓ Down	← Left	→ Right
S0 (Start)	0	0	0	0
S1	0	0	0	0
S2	0	0	0	0
S3	0	0	0	0
S4	0	0	0	0
S5	0	0	0	0
S6 (Goal)	0	0	0	0

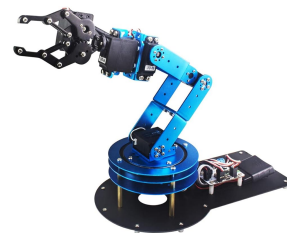
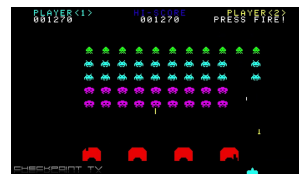
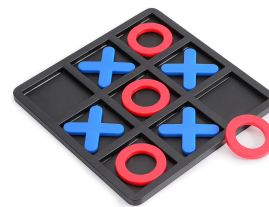


Recap: Q-Learning

Our Q-table worked perfectly for a 3×3 maze and tic-tac-toe. What happens when the problem gets bigger?

Recap: Q-Learning

Problem	Number of States	Q-table size
3×3 Maze	7 states	$7 \times 4 = 28$ values
Tic-tac-toe	~5,478 states	~22,000 values
Chess	$\sim 10^{47}$	impossible
Atari game (pixels)	$\sim 10^{170}$	impossible
Robot arm (continuous)	infinite	impossible



Recap: Q-Learning

"Can we build a table with 10^{47} rows? How much RAM would that need?"

This creates the natural question: "*So what do we do instead?*" — and **DQN** is the answer.

Deep Q-Network (DQN)

Deep RL Value Methods

- DQN (DeepMind, 2013) uses a deep neural network to approximate $Q(s,a)$, enabling RL on high-dimensional inputs like raw pixel images.

- **Neural Network Approximator**

Replace Q-table (discrete) with a deep CNN. Input: raw pixels/state; Output: Q-value for each action.

- **Target Network**

Use a separate, slowly-updated copy of the Q-network to generate stable TD targets. Prevents deadly oscillations.

- **Experience Replay Buffer**

Store past transitions (s, a, r, s') in memory. Sample random mini-batches to break temporal correlations and stabilize training.

- **Achievement**

DQN achieved human-level or superhuman performance on 49 Atari games using only raw pixel input — a landmark result in deep RL.

Deep Q-Network (DQN)

The Big Idea of DQN

- Instead of looking up $Q(s,a)$ in a table, we APPROXIMATE $Q(s,a)$ using a neural network.

- **Q-Learning:**

Q-table

$s \rightarrow$ look up row $\rightarrow$ Q values for each action

- **DQN:**

Neural Network

$s \rightarrow$ feed forward $\rightarrow$ Q values for each action

The Bellman equation stays exactly the same. Only the storage and computation of Q changes.

Deep Q-Network (DQN)

The Analogy That Makes It Click

- **Q-Learning:**

Q-table is like a phone book — you look up each entry individually. Works fine for a small city. Useless for the entire world.

- **DQN:**

Neural network is like Google — it has never seen your exact query before, but it generalizes from patterns to give you a good answer instantly.

- The neural network **generalizes** across similar states. If it learns that "being close to the goal is good" in one part of the maze, it applies that knowledge to similar situations it has never seen before.

Deep Q-Network (DQN)

What Changes from Q-Learning to DQN

Component	Q-Learning	DQN
Q-value storage	Table (exact)	Neural network (approximate)
Update method	Bellman equation directly	Gradient descent on loss function
Input	State index	State features / pixels
Output	$Q(s,a)$ for one pair	$Q(s,a)$ for ALL actions at once
Handles large spaces?	Handles large spaces?	✓ Yes
Bellman equation	✓ Same	✓ Same
ϵ -greedy policy	✓ Same	✓ Same
Reward signal	✓ Same	✓ Same

Deep Q-Network (DQN)

The Two Key DQN Innovations

- **Innovation 1: Experience Replay**

The problem: If the agent trains on consecutive steps, the data is highly correlated — the network overfits to recent experience and forgets old knowledge.

The solution: Store all experiences in a **replay buffer** and train on random batches.

- Every step $\rightarrow$ store (s, a, r, s') in replay buffer

Every training step $\rightarrow$ sample random batch of 32 experiences
 $\rightarrow$ train network on that batch

Deep Q-Network (DQN)

The Two Key DQN Innovations

- **Innovation 2: Target Network**

The problem: The network is chasing a moving target. Every time you update the network, the target Q-value also changes — like trying to hit a target that moves every time you shoot.

The solution: Keep TWO networks:

-

Online Network → updated every step (the learner)

Target Network → frozen, copied from online every N steps (the stable target)

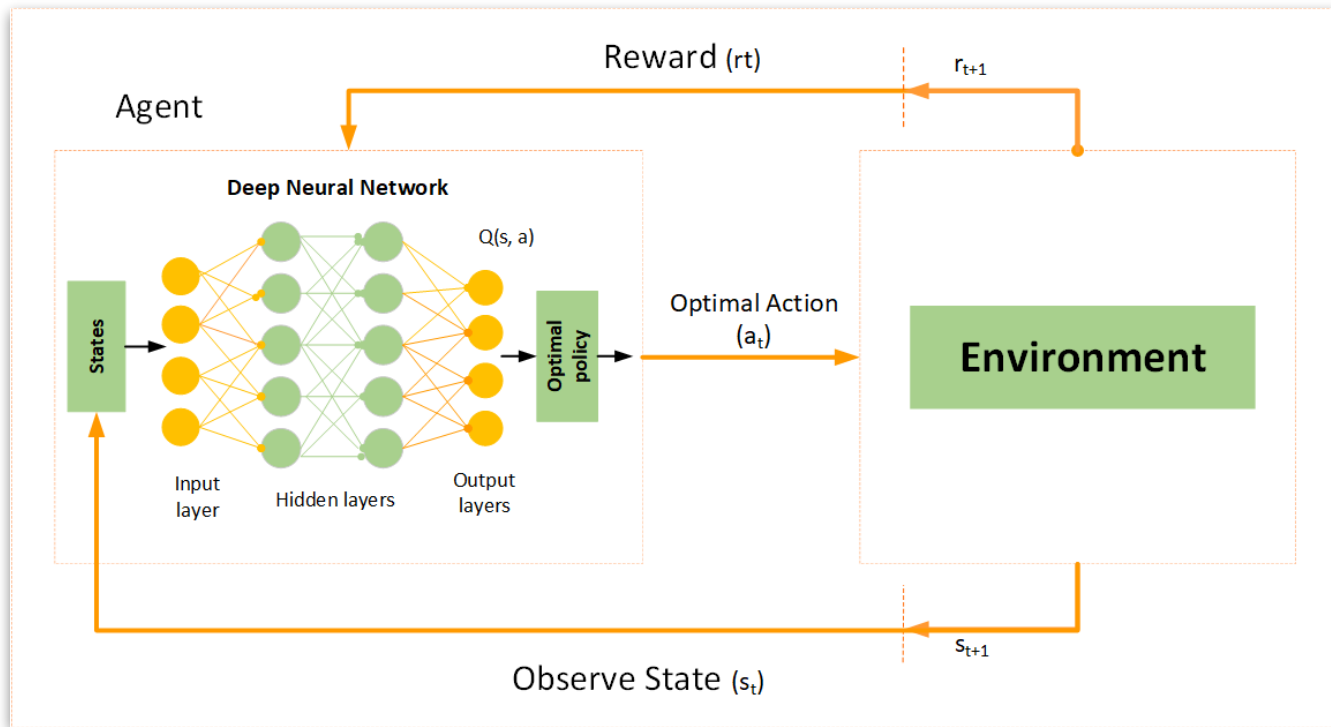
Deep Q-Network (DQN)

Loss function of DQN

$$\mathcal{L} = \left[\underbrace{R + \gamma \max_{a'} Q_{target}(s', a')}_{\text{target (frozen)}} - \underbrace{Q_{online}(s, a)}_{\text{prediction}} \right]^2$$

Deep Q-Network (DQN)

Architecture of DQN



Deep Q-Network (DQN)

Key Idea: Instead of storing $Q(s,a)$ in a table, approximate it using a Neural Network.

Q-table: state index $\rightarrow$ look up row $\rightarrow [Q(s,\uparrow), Q(s,\downarrow), Q(s,\leftarrow), Q(s,\rightarrow)]$

DQN: state vector $\rightarrow$ neural net $\rightarrow [Q(s,\uparrow), Q(s,\downarrow), Q(s,\leftarrow), Q(s,\rightarrow)]$

We use the same 3×3 maze. To keep the hand calculation manageable, we design a minimal network:

• S 	•	• W
•	• W	•
•	•	• G 

Deep Q-Network (DQN)

Architecture of DQN

We use the same 3×3 maze. To keep the hand calculation manageable, we design a minimal network:

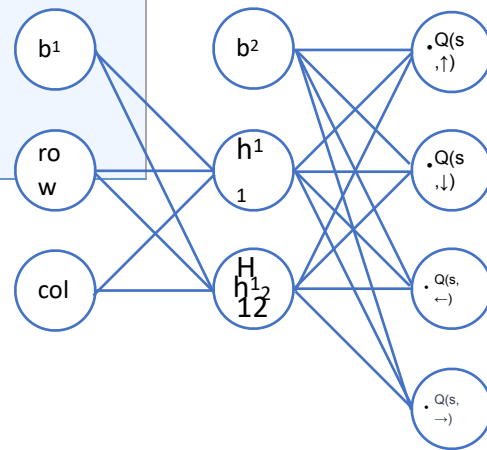
Architecture: Input(2) → Hidden(2, ReLU) → Output(4, Linear)

Input : [row, col] — the agent's position as 2 numbers

Hidden : 2 neurons with ReLU activation

Output : 4 neurons = $Q(s, \uparrow)$ $Q(s, \downarrow)$ $Q(s, \leftarrow)$ $Q(s, \rightarrow)$

$\text{ReLU}(x) = \max(0, x)$ — simply sets any negative value to zero.

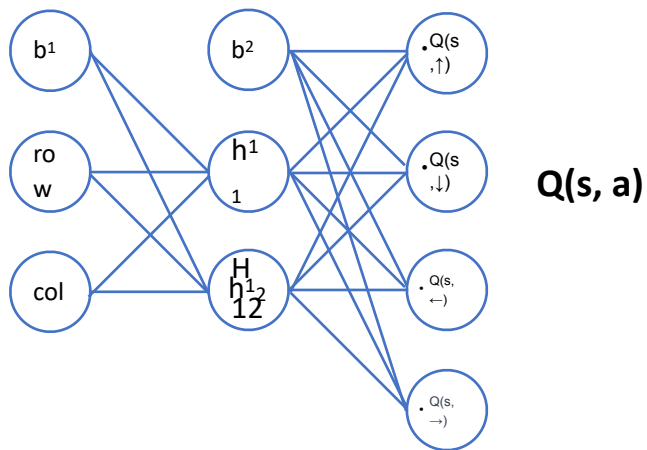


Deep Q-Network (DQN)

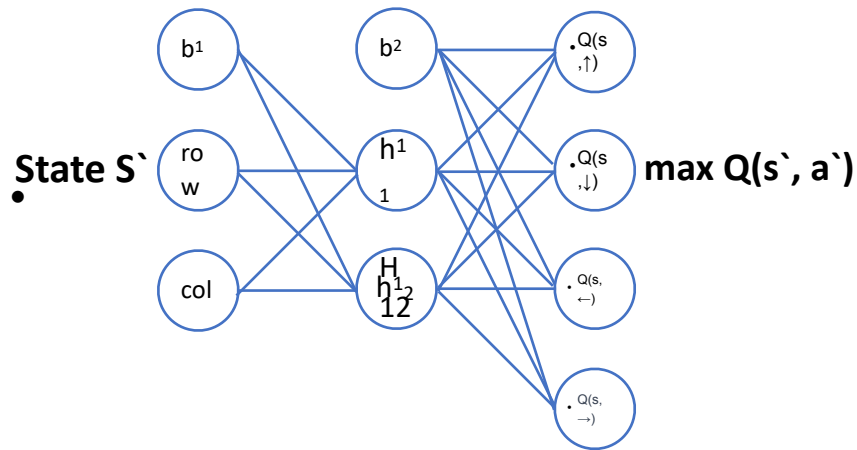
Architecture of DQN

- **Online Network** → updated every step (the learner)
- **Target Network** → frozen, copied from online every N steps (the stable target)

Online Network



Target Network



Thank You!

Introduction to Reinforcement Learning

Coming Up:

- Deep RL: CNN + RNN policies
- OpenAI Gymnasium hands-on lab
- Implement Q-Learning from scratch
- Train your first PPO agent

Questions?

Dr. Oybek Eraliev
CSE4020 — Capstone Design